

TRABALHO FINAL DE POO: SPOTIFY - PRIMEIRA ENTREGA

Integrantes do grupo:

1. Daniel Mendonça Garcia - 202504988
2. Gean Pablo Pereira Camargo - 202501707
3. Paulos César Ribeiro Soares - 202501291

Link do Repositório no GitHub:

https://github.com/daniel-mg-br/projeto_spotify

Na seção Read Me, há a explicação das hierarquias de pacotes e sobre onde está cada classe, juntamente com a definição do projeto e os requisitos. Na pasta 'media', estão contidos os png dos diagramas de caso de uso e de classe separadamente. Os diagramas também estão nas últimas páginas deste documento. Na pasta 'docs' estão localizadas a documentação Javadoc e este documento.

Definição do Problema

Nos serviços de streaming atuais (Spotify, Deezer, SoundCloud, etc) gerenciar as próprias músicas e o perfil pode ser uma tarefa cansativa, devido à alta poluição visual logo na tela de entrada, que contém: dezenas de recomendações de mixes de músicas - de gêneros que muitas vezes a pessoa não se identifica - juntamente com anúncios e músicas "parecidas" com as suas. Pensando que esse desconforto pode ser estendido também para os criadores de conteúdo, desenvolvemos este projeto.

Nosso trabalho final é um sistema de gerenciamento de usuários, podendo ser ouvintes ou artistas, e conteúdos – consistindo de músicas, álbuns, playlists e podcasts – . Dispensando a funcionalidade de player de músicas e podcasts, aqui o usuário pode manipular e consultar livremente os conteúdos que consome e cria nos aplicativos de streaming, com uma visualização limpa e intuitiva sem o bombardeio de informações dos serviços convencionais.

Em poucas palavras, para evitar distrações e cansaço visual, o usuário apenas registra o conteúdo que escuta ou cria, e pode manipular livremente esse catálogo.

Tecnologias Utilizadas

- PlantUML e AstahUML para organização de diagramas;
- Eclipse IDE para o desenvolvimento do projeto;
- Linguagem Java, versão 25 (JDK 25);
- JUnit 4 para testes unitários;
- Java Swing para interface gráfica do usuário;
- Banco de Dados SQLite 3 para persistência de dados;
- JDBC para conexão com banco de dados;
- Git e GitHub para versionamento de código.

Lista de Requisitos

- Adicionar Usuários: adicionar registros de Ouvintes e Criador;
- Remover Usuários: remover registros de Ouvintes e Criador;
- Consultar Usuários: consultar dados de registros dos Usuários;
- Adicionar músicas ao banco de músicas do sistema;
- Remover músicas do banco de músicas do sistema;
- Criar playlists para os Usuários escutarem;
- Criar álbuns para os Artistas “lançarem”;
- Adicionar músicas disponíveis às playlists dos Usuários;
- Remover músicas nas playlists dos Usuários;
- Adicionar músicas aos álbuns dos Criadores;
- Remover músicas nos Álbuns dos Criadores;
- Consultar playlists e álbuns;

Casos de Uso

1. Ator: Usuário - Representa o usuário real do sistema, seja ele Administrador, Criador ou Ouvinte.

- Alterar dados pessoais;
- Consultar dados.

2. Ator: Administrador - Responsável pela gestão global do sistema, incluindo contas e o acervo musical.

- Herda do Ator Usuário;
- Suspender usuários;

- Consultar usuários;
- Consultar conteúdos;
- Remover conteúdos.

3. Ator: Ouvinte - Representa o ouvinte comum do sistema.

- Criar e excluir playlists;
- Adicionar músicas disponíveis às playlists;
- Remover músicas das playlists;
- Consultar playlists e álbuns/podcasts;
- Adicionar e remover playlists e álbuns/podcast favoritos;
- Compartilhar playlists.
- Herda do Ator Usuário;

4. Ator: Criador - Representa o criador de conteúdo.

- Herda do ator Usuário;
- Criar / excluir álbuns e podcasts;
- Adicionar músicas aos álbuns e episódios aos podcasts;
- Remover músicas dos Álbuns e episódios dos podcasts;
- Lançar álbuns.

Detalhamento de Pacotes e Classes (src)

Pacote Model: contém as classes “modelo” do sistema

- **Subpacote Actors (Atores):**
 - **Conta:** representa a conta vinculada ao usuário;
 - **Usuario:** classe abstrata com os dados gerais do usuário do sistema;
 - **Administrador:** filha de Usuario, representa um administrador do sistema, moderando usuários e conteúdos;
 - **Ouvinte:** filha de Usuario, representa o ouvinte comum do sistema;
 - **Criador:** filha de Usuario, representa o criador de conteúdo comum.
- **Subpacote Content (Conteúdos):**
 - **Conteudo:** classe abstrata contendo os dados gerais de um conteúdo registrado;
 - **Musica:** filha de Conteudo, representa as músicas de um Criador;
 - **Episodio:** filha de Conteudo, representa os episódios de um Podcast;
 - **Playlist:** representa playlists que o Ouvinte pode criar e manipular;
 - **Album:** representa os álbuns lançados por um Criador;

- **Podcast:** representa os podcasts mantidos por um Criador.

Pacote Dao: pacote contendo as classes responsáveis por manipular o banco de dados (Data Access Object)

- **ConnectionFactory:** classe responsável por estabelecer a conexão JDBC com o SQLite, retornando um objeto de Connection para as demais classes DAO. Implementada com o padrão de projeto Factory Method.
- **Classes DAO para os usuários do sistema:**
 - ContaDAO;
 - AdministradorDAO;
 - OuvinteDAO;
 - CriadorDAO.
- **Classes DAO para os conteúdos do sistema:**
 - MusicaDAO;
 - EpisodioDAO;
 - PlaylistDAO;
 - AlbumDAO;
 - PodcastDAO.
- As classes DAO permitem a manipulação de dados de usuários e conteúdos, com as operações clássicas de CRUD: consultar, inserir, atualizar e deletar.

Pacote Controller: pacote com as classes intermediárias entre o usuário do sistema e a manipulação de dados. Oferece as funcionalidades que os usuários podem utilizar.

- **ControllerAutenticador:** responsável por realizar o cadastro, login e logout de usuários.
- **ControllerPerfil:** engloba as funcionalidades de edição de dados do perfil, tais como: mudança de nome, sexo e senha do usuário, além da atualização do plano da conta.
- **ControllerAdmin:** oferece a remoção de conteúdo e suspensão de usuários para o administrador do sistema.

- **ControllerOuvinte:** contém as funcionalidades de criação, edição e compartilhamento de playlists, junto da adição e remoção de álbuns e podcasts favoritos para o ouvinte do sistema.
- **ControllerCriador:** é composto pelos métodos de manipulação de álbuns e podcasts do criador de conteúdo.

Pacote View: constitui as classes responsáveis pela interface gráfica (GUI) da aplicação.

- **CadastroGUI:** tela para o cadastro de novos usuários;
- **LoginGUI:** tela para o login de usuários existentes;
- **MenuAdminGUI:** tela para as operações referentes ao administrador;
- **MenuCriadorGUI:** tela para as operações referentes ao criador de conteúdo;
- **MenuOuvinteGUI:** tela para as operações referentes ao ouvinte.

Pasta auxiliar Test: pasta com as classes de testes unitários (JUnit) para as classes do pacote 'model': ContaTest, CriadorTest, MusicaTest, OuvinteTest, PlaylistTest, PodcastTest, UsuarioTest e AdministradorTest.

Pasta auxiliar Media: pasta com a documentação do projeto, incluindo o pdf deste trabalho escrito, e uma subpasta com a documentação usando Javadoc.

Diagramas referentes ao projeto

A seguir, estão os diagramas de caso de uso e os diagramas de classe que definem os conceitos e relações implementados neste trabalho. Para os diagramas de classe, decidimos fazer três:

- **Diagrama de domínio:** engloba as classes conceituais do pacote model, focando nas relações de herança, associações e agregações;
- **Diagrama de negócio:** foca nas dependências e associações entre as classes relacionadas à interface gráfica e as classes do pacote controller;
- **Diagrama completo:** diagrama com todas as classes, descrevendo o funcionamento do sistema.

Diagrama de Casos de Uso

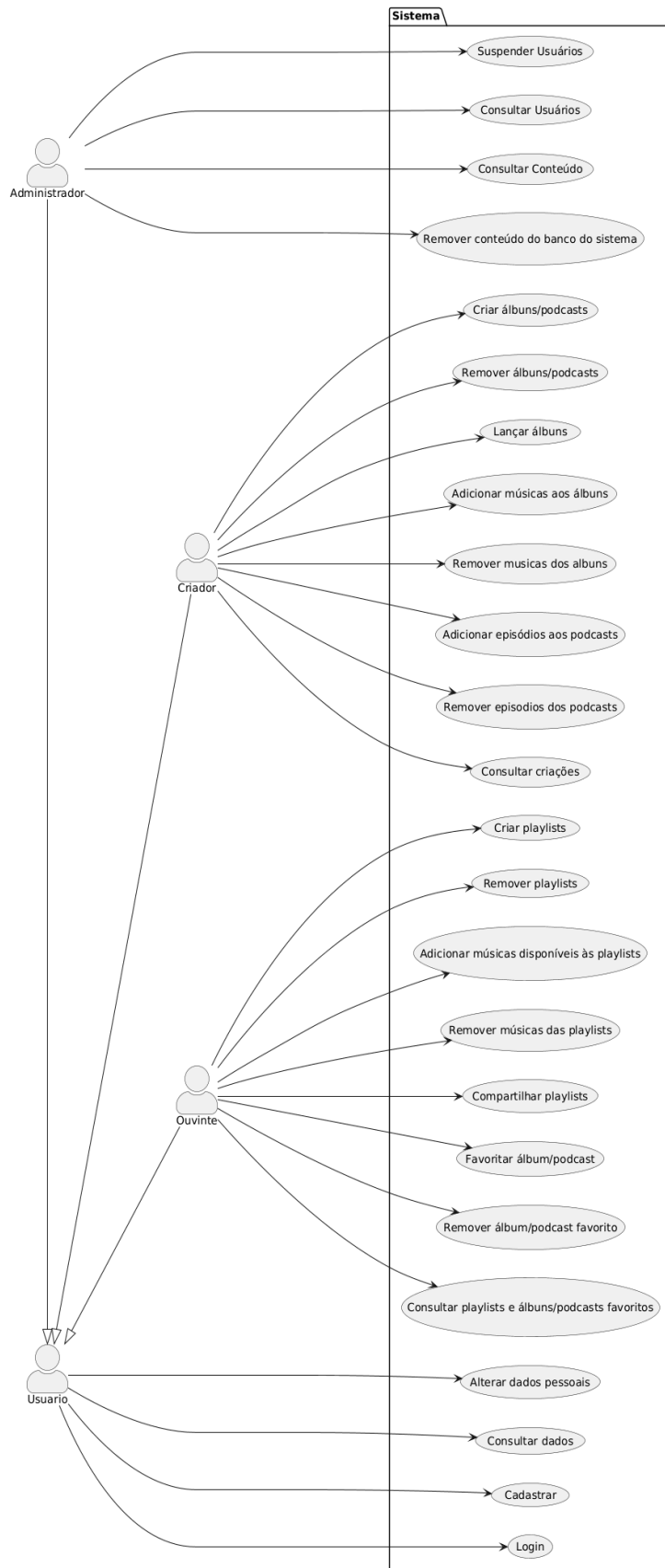


Diagrama de Classes de Domínio

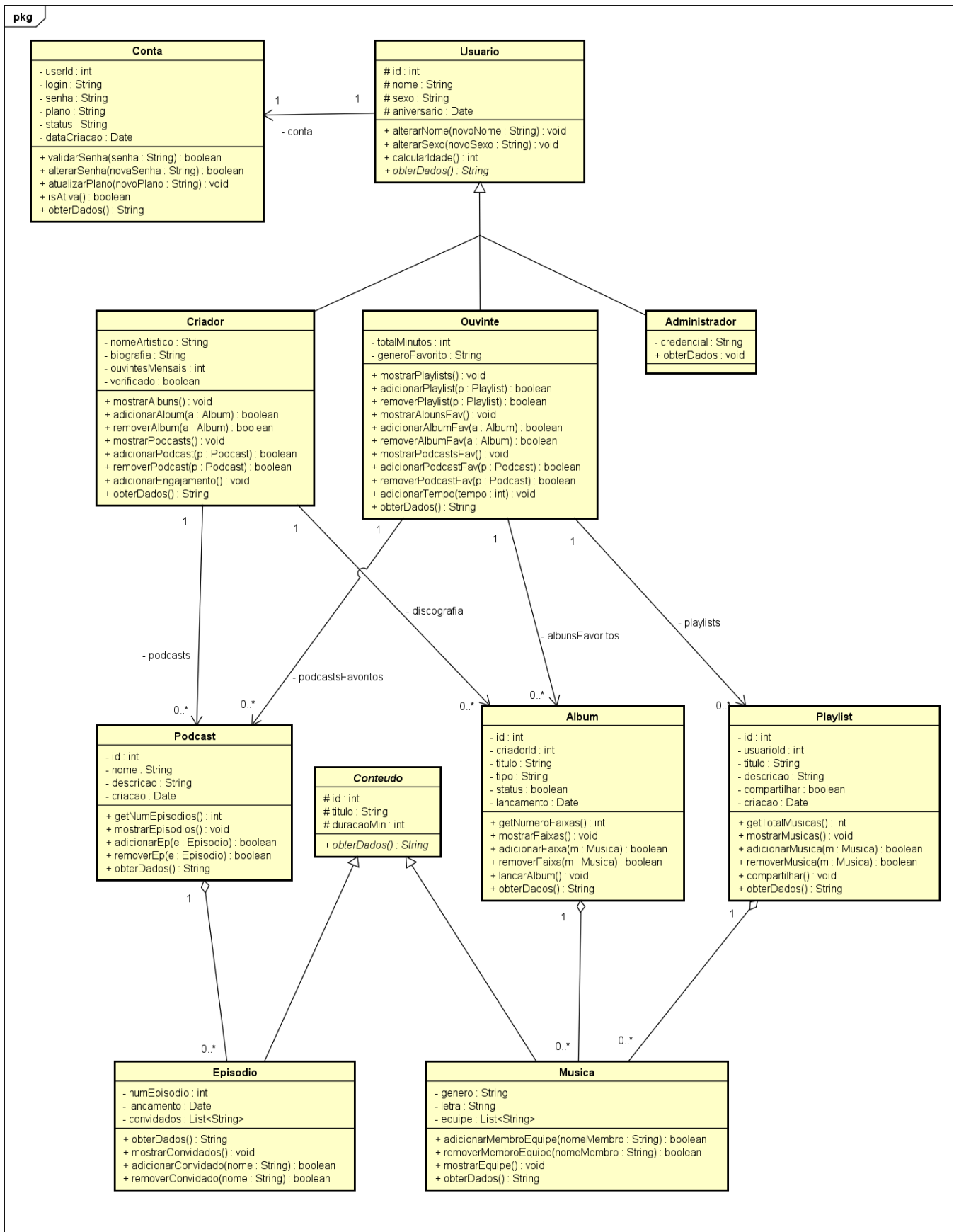


Diagrama de Negócio

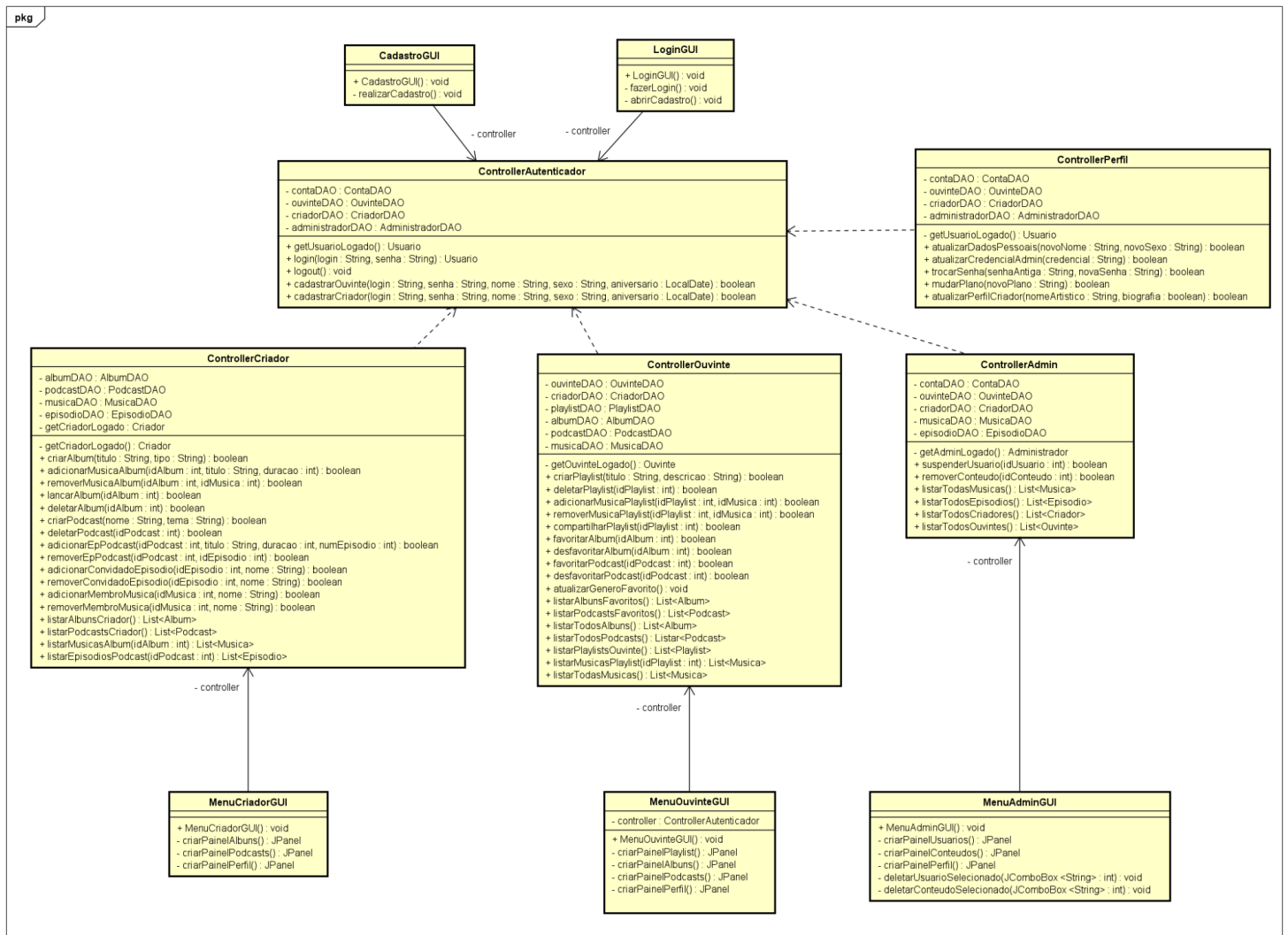


Diagrama Completo

